

Análise Técnica: Testes Unitários no NestJS

Entendendo a anatomia do `CreateTeamMemberService.spec.ts`

1. O Coração do Teste: `TestingModule`

No NestJS, não instanciamos classes manualmente com `new Service()`. Usamos o `Test.createTestingModule`. Isso simula o **Inversion of Control (IoC)** do framework.

Por que usar `providers`?

Ao declarar o `CreateTeamMemberService` e o mock do `TeamMembersRepository`, você está dizendo ao Nest: *"Quando o serviço pedir o repositório no construtor, entregue este objeto falso (mock) que eu criei"*.

2. Mocks vs. Implementações Reais

Um teste unitário deve testar **apenas** a lógica da classe. Se usássemos o repositório real, estaríamos testando o banco de dados (Teste de Integração).

```
useValue: {  
  create: jest.fn(),  
  findByName: jest.fn(),  
  ...  
}
```

Aqui, `jest.fn()` cria uma função "espiã". Ela não faz nada por padrão, mas permite que você:

- Simule retornos específicos (`mockResolvedValue`).
- Verifique se foi chamada (`toHaveBeenCalled`).

3. Setup: `beforeEach`

O `beforeEach` garante que cada teste comece com uma folha em branco. Isso evita o **State Leakage** (quando o resultado de um teste afeta o próximo).

Dica Técnica: O uso de `jest.Mocked<TeamMembersRepository>` ajuda o TypeScript a entender que aquele repositório agora possui métodos extras do Jest, como o `.mockResolvedValue()`.

4. Anatomia dos Casos de Teste

Caso 1: Sucesso (Happy Path)

Neste teste, você configura o cenário ideal: o nome não existe no banco (`null`) e o banco retorna o objeto criado.

As asserções (`expect`) verificam o **comportamento**:

- O serviço consultou o nome antes de criar? (Segurança)
- Ele enviou os dados corretos para o banco? (Integridade)

Caso 2: Erro de Negócio (Edge Case)

Este é o teste mais importante. Ele garante que sua regra de **nomes únicos** funciona.

```
await expect(promise).rejects.toBeInstanceOf(AppError);
```

Aqui testamos se a exceção correta é lançada. Note o uso de `not.toHaveBeenCalled()` no final: isso prova que, se o nome já existe, o código para ali e **nunca** chega a chamar a função de criação.

5. Resumo de Comandos

- `mockResolvedValue`: Define o que a função assíncrona retorna.
- `toHaveBeenCalled`: Valida se os argumentos passados estão corretos.
- `toBeInstanceOf`: Garante que o erro é do tipo que sua arquitetura espera (ex: `AppError`).

